

Divide and Conquer Algorithm Visualizer

First Phase Project Report

Design and Analysis of Algorithms (DAA)

Web-Based Visualization Tool for Divide-and-Conquer Algorithms

Algorithms Covered:

Merge Sort, Quick Sort, Binary Search, Closest Pair of Points (Divide & Conquer), and Strassen's Matrix Multiplication

Hosted at:

<https://aryansingh0070.github.io/AlgoNexus/>

Submitted By:

Aryan Singh Sikarwar (24BCE0403) and Aryan Yadav (24BCT0306)

Submitted for academic evaluation · Original work

1. Introduction

One of the persistent challenges in teaching algorithms is bridging the gap between a written algorithm and an intuitive understanding of how it executes. This challenge becomes even more significant for recursive algorithms, where the dynamic unfolding of nested calls, sub-problem decomposition, and return values can be difficult to visualize using only pseudocode or static diagrams. Students may read a recurrence such as $T(n) = 2T(n/2) + O(n)$ and compute its complexity, but that alone rarely provides a clear mental picture of how recursive calls are structured, how deep the recursion tree grows, or how partial results combine to produce the final solution.

The Recursion Tree Visualizer is a web-based educational tool developed as a project for the Design and Analysis of Algorithms (DAA) course. The system aims to address this conceptual gap by rendering the execution of recursive algorithms as interactive and animated recursion trees. Through step-by-step visualization, the tool makes recursive calls visible and helps learners understand how divide-and-conquer algorithms decompose problems into smaller subproblems and recombine their results. The tool runs directly in the browser without requiring installation, making it accessible across different devices and platforms.

Five divide-and-conquer algorithms are implemented in the system: Merge Sort, Quick Sort, Binary Search, the Closest Pair of Points algorithm, and Strassen's Matrix Multiplication. Merge Sort demonstrates how a problem can be divided into two equal subproblems and merged to produce a sorted result with $O(n \log n)$ complexity. Quick Sort illustrates the partitioning approach, where elements are organized around a pivot and recursively sorted. Binary Search demonstrates efficient searching within a sorted array by repeatedly halving the search space, achieving $O(\log n)$ complexity. The Closest Pair of Points algorithm illustrates geometric divide-and-conquer techniques used to find the minimum distance between points in a plane with $O(n \log n)$ time complexity. Strassen's Matrix Multiplication demonstrates advanced divide-and-conquer for matrix operations, achieving sub-cubic $O(n^{2.807})$ complexity through recursive 2×2 block decomposition.

By visualizing the recursion trees generated by these algorithms, students can directly observe how recursion depth, branching factor, and problem size influence algorithmic complexity. The system also enables step-by-step navigation with rich tooltips and contextual side-panel descriptions explaining the purpose of each recursive call, smooth visual transitions, and color-coded node states distinguishing visited, pending, and active nodes. The interface remains focused so that attention remains on the structure of the recursion tree and the algorithmic relationships it represents.

1.1 Problem Statement

Recursion and divide-and-conquer strategies are central to the Design and Analysis of Algorithms curriculum, yet students consistently report them among the most difficult topics to internalize. The core difficulty is representational: the standard tools available to students describe algorithmic structure but do not make it visible. When a student reads that algorithms such as Merge Sort or Quick Sort divide a problem into smaller subproblems, they understand the theoretical concept but often fail to develop a clear mental model of how those recursive calls actually unfold during execution. It can be difficult to visualize how many nodes appear in the recursion tree, how the recursion

depth grows with input size, and how partial solutions are combined to produce the final result.

The Recursion Tree Visualizer directly addresses this representational gap by providing an interactive environment where the recursive structure of divide-and-conquer algorithms becomes visible. By visualizing recursion trees for algorithms such as Merge Sort, Quick Sort, Binary Search, the Closest Pair of Points, and Strassen's Matrix Multiplication, the tool allows students to observe how problems are repeatedly divided, how recursion progresses step by step, and how solutions propagate back up the recursion tree.

1.2 Objectives

- To visualize the step-by-step execution of Merge Sort, Quick Sort, Binary Search, the Closest Pair of Points algorithm, and Strassen's Matrix Multiplication as animated and navigable recursion trees.
- To help students understand the relationship between an algorithm's recurrence structure and its asymptotic complexity by making the recursion tree's depth and branching factor directly observable.
- To provide interactive navigation that enables self-paced and active exploration of recursive algorithms rather than passive viewing.
- To deliver annotated step-by-step explanations through tooltips and contextual side panels, describing the purpose of each recursive call rather than just displaying values.
- To apply smooth visual transitions and color-coding that clearly distinguish between visited, pending, and active nodes throughout the recursion tree.
- To deliver these capabilities through a zero-installation, browser-based interface accessible without any setup or dependencies.
- To serve as a supplementary learning tool for Design and Analysis of Algorithms (DAA) coursework, examination preparation, and classroom demonstration.

1.3 Modifications from Abstract

The original project abstract proposed the visualization of Merge Sort, Quick Sort, Binary Search, and the Closest Pair of Points algorithm. Based on feedback received and a reassessment of scope, the implementation has been extended and refined in three significant ways:

1. Strassen's Matrix Multiplication added: The algorithm set now includes Strassen's Matrix Multiplication, which demonstrates a fundamentally different divide-and-conquer pattern — recursive 2×2 block decomposition with only 7 multiplications instead of 8. Its recursion tree has a branching factor of 7 and achieves sub-cubic complexity $O(n^{2.807})$, offering a contrasting perspective to sorting and searching algorithms.

2. Contextual tooltips and side panel descriptions added: Each recursive step now includes descriptive tooltips and a side panel explaining the purpose of that specific recursive call — why the subproblem is being created, what values are being processed, and how the current step fits into the overall algorithm. This moves beyond mere value display toward conceptual annotation.

3. Enhanced visual transitions and node color-coding: Smooth animated transitions are applied between recursive steps. A consistent color-coding scheme now distinguishes

three node states: active (currently executing call, highlighted in blue), visited (completed calls, shown in green), and pending (calls not yet reached, shown in grey). This visual differentiation helps students track algorithm progress at a glance.

By visualizing these algorithms side by side, students can observe how different divide-and-conquer strategies influence recursion depth, branching patterns, and overall time complexity.

2. Literature Review

ABSTRACT

This literature review examines the research foundations underlying the design and educational rationale of the Recursion Tree Visualizer. The review covers three themes: (1) the empirical effectiveness of algorithm visualization tools in CS education; (2) the specific pedagogical challenges of teaching recursion; and (3) the algorithmic theory and educational significance of the divide-and-conquer algorithms implemented, namely Merge Sort, Quick Sort, Binary Search, Closest Pair of Points, and Strassen's Matrix Multiplication. The review identifies gaps in existing tooling and concludes that the Recursion Tree Visualizer occupies a well-defined niche by providing browser-accessible, step-by-step recursion tree animation with contextual annotations.

2.1 Background and Context

The teaching of algorithms and data structures is widely regarded as one of the most cognitively demanding areas in undergraduate computer science education. Algorithms describe dynamic, abstract computational processes that resist comprehension through static, textual descriptions alone. This challenge is particularly acute for recursive algorithms, where students must simultaneously hold multiple nested call states in working memory while tracking how values propagate back up through the call chain.

Over two decades of research has consistently found that algorithm visualization (AV) tools offer measurable pedagogical benefits over conventional instruction when used correctly. Hundhausen, Douglas, and Stasko (2002), in a landmark meta-study of 24 experimental studies, concluded that the most effective deployments of AV technology were those that actively engaged students through prediction tasks, what-if analyses, and interactive navigation rather than passive animation viewing. This finding directly justifies the step-by-step navigation and contextual annotation design of the Recursion Tree Visualizer.

2.2 Scope of the Review

This review examines research across four intersecting domains: (1) the general effectiveness of algorithm visualization tools in CS education; (2) the specific pedagogical challenges of teaching recursion and divide-and-conquer strategies; (3) the algorithmic theory and educational literature pertaining to key divide-and-conquer algorithms; and (4) design principles for web-based interactive learning environments.

2.3 Research Objectives

- To establish the empirical basis for using visual and interactive tools in algorithm education, with a focus on recursion-specific visualization tools.
- To survey and critically evaluate existing algorithm visualization platforms in order to identify their capabilities and limitations relative to the Recursion Tree Visualizer.
- To review the algorithmic and educational literature on key divide-and-conquer algorithms including Merge Sort, Quick Sort, Binary Search, Closest Pair of Points, and Strassen's Matrix Multiplication.

- To identify gaps in existing visualization tools and educational literature that the Recursion Tree Visualizer aims to address.

2.4 Methodology (Search Strategy)

This literature review follows a structured search strategy designed to ensure comprehensive coverage of relevant peer-reviewed research, conference publications, and educational resource literature. The search was conducted between March 2025 and March 2026, drawing on academic databases, university course materials, and reputable educational platforms. A total of 34 sources were initially identified; after applying predefined inclusion and exclusion criteria, 22 sources were retained for detailed analysis.

Criterion	Details
Publication period	1990–2026; exceptions for seminal earlier works (Merge 1968 – Knuth, Quick Sort 1960 – Hoare)
Language	English-language publications only
Source types	Peer-reviewed journals, ACM/IEEE proceedings, university course materials, established educational platforms
Relevance	Must directly address algorithm visualization, recursion pedagogy, divide-and-conquer algorithms, or web-based educational tool design
Excluded	K–12 studies; sorting/graph-only tools unrelated to recursion trees; non-English publications; grey literature without academic attribution

2.5 Theme 1: Effectiveness of Algorithm Visualization in CS Education

The use of visualization as a pedagogical tool in computer science education has been studied extensively since the late 1980s. The earliest influential framework came from Brown and Sedgewick's Balsa system (1984), which pioneered the concept of software that animated algorithmic execution in real time. Hundhausen et al. (2002) aggregated findings from 24 experimental studies and concluded that active engagement — constructing animations, making predictions, navigating step-by-step — consistently outperformed passive viewing. This finding directly justifies both the step-by-step navigation design and the contextual annotation features of the Recursion Tree Visualizer.

A 2024 study published in Educational Technology Research and Development (Springer) evaluated a custom web-based animation platform for undergraduate Data Structures courses and found that approximately 95% of students achieved grades above 80 on post-visualization assessments, with high user acceptance scores. A VR-based algorithm visualization study (Academia.edu, 2025) reported that 76.9% of students exposed to interactive visualizations performed better on algorithmic tasks compared to control groups.

Visualization tools help students connect theoretical complexity analysis with actual algorithm behavior. When recursion trees are displayed visually, students can directly observe how problem size decreases at each recursive level and how the number of

subproblems grows. For Strassen's Matrix Multiplication, visualization makes the branching factor of 7 (versus the naive 8) immediately apparent — a fact that is difficult to appreciate from pseudocode alone.

2.6 Theme 2: Recursion Visualization — Challenges and Existing Tools

Recursion is consistently identified in the CS education literature as one of the most cognitively demanding topics for undergraduate students. Miller and Ranum advocate tree diagrams as one of the most effective representational forms for explaining recursion, since recursion trees simultaneously display the stack of pending calls and the order in which results propagate back up the call chain.

Several existing visualization platforms attempt to address aspects of this problem. VisuAlgo (Halim, 2011–present) is comprehensive but its recursion visualization primarily focuses on generic procedures rather than providing detailed recursion tree representations with step-level annotations. SRec (Velázquez-Iturbide et al., 2008) is pedagogically grounded but is implemented as a Java desktop application, which creates deployment friction. Python Tutor (Guo, 2013) operates at the code level rather than the algorithmic level and does not emphasize structural decomposition. None of these tools provide smooth animated transitions with color-coded node states (visited / active / pending), nor do they offer tooltips or side-panel descriptions explaining the semantic purpose of each recursive call.

2.7 Theme 3: Divide-and-Conquer Algorithms — Theory and Educational Significance

Divide-and-conquer is one of the most fundamental algorithm design paradigms taught in DAA courses. Each of the five algorithms implemented demonstrates a distinct recursive pattern:

- Merge Sort — balanced binary recursion, $T(n) = 2T(n/2) + O(n)$, $O(n \log n)$
- Quick Sort — partition-based recursion, average $T(n) = 2T(n/2) + O(n)$, $O(n \log n)$ average, $O(n^2)$ worst
- Binary Search — logarithmic single-branch recursion, $T(n) = T(n/2) + O(1)$, $O(\log n)$
- Closest Pair of Points — geometric divide-and-conquer, $T(n) = 2T(n/2) + O(n)$, $O(n \log n)$
- Strassen's Matrix Multiplication — 7-subproblem block recursion, $T(n) = 7T(n/2) + O(n^2)$, $O(n^{2.807})$

Strassen's algorithm, introduced by Volker Strassen in 1969, was the first algorithm to break the $O(n^3)$ barrier for matrix multiplication. It decomposes two $n \times n$ matrices into 2×2 block matrices and computes 7 matrix products (M_1 – M_7) instead of 8, reducing the branching factor in the recursion tree. Educational resources at MIT OpenCourseWare and Cornell CS3110 cite Strassen's algorithm as an important example of how clever problem reformulation can reduce algorithmic complexity, making it an excellent candidate for recursion tree visualization.

2.8 Gaps Identified

1. Absence of a dedicated web-based recursion tree visualizer for multiple divide-and-conquer algorithms — existing platforms animate algorithms but rarely provide annotated, step-level recursion tree exploration.

2. Deployment friction — the most pedagogically grounded recursion tool (SRec) requires Java installation; few modern browser-based systems are designed specifically for algorithm analysis.
3. Lack of multi-algorithm comparative visualization — most tools visualize algorithms individually; none allow comparison including sub-cubic algorithms like Strassen's.
4. Missing contextual annotation — existing tools display values but rarely explain the semantic purpose of each recursive call.
5. Insufficient visual differentiation — few tools apply consistent color-coding schemes to distinguish visited, pending, and active nodes with smooth transitions.

2.9 References

[1] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages and Computing*, vol. 13, no. 3, pp. 259–290, 2002.

[2] J. Á. Velázquez-Iturbide, A. Pérez-Carrasco, and J. Urquiza-Fuentes, "SRec: An animation system of recursion for algorithm courses," *ACM SIGCSE Bulletin*, vol. 40, no. 3, pp. 225–229, 2008.

[3] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," *Proc. ACM SIGCSE Technical Symposium*, 2013.

[4] S. Halim, VisuAlgo, National University of Singapore, 2011–present. <https://visualgo.net/>

[5] B. Miller and D. Ranum, *Problem Solving with Algorithms and Data Structures*, Runestone Academy, 2011.

[6] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.

[7] V. Strassen, "Gaussian Elimination is not Optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.

[8] MIT OpenCourseWare, 6.006 Introduction to Algorithms. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/>

[9] Cornell University, CS3110 – Recursion Trees and the Master Method. <https://cs3110.github.io/>

[10] GeeksforGeeks, "Strassen's Matrix Multiplication," 2023. <https://www.geeksforgeeks.org/strassens-matrix-multiplication/>

3. Design Figure

The following figure presents the complete system architecture of the Recursion Tree Visualizer. The design adopts a three-layer model: User Interface, Application Logic, and Rendering, which separates concerns clearly and reflects the unidirectional data flow governing the tool's operation. Each layer communicates with adjacent layers through well-defined interfaces: user events flow downward from the interface into the logic layer, while rendered output flows upward from the rendering layer back to the browser display.

[Figure 1: System architecture diagram — three-layer model showing User Interface Layer (Algorithm Selector, Input Panel, Navigation Controls, Side Panel/Tooltip Display, Display Canvas), Application Logic Layer (Merge Sort Engine, Quick Sort Engine, Binary Search Engine, Closest Pair Engine, Strassen Engine, Tree State Manager), and Rendering Layer (Tree Layout Engine, Node Renderer, Step Animator with smooth transitions and color-coding)]

[Figure 2: Recursion tree visualization for the Merge Sort algorithm on the array [38, 27, 43, 3, 9, 82, 10], showing visited (green), active (blue), and pending (grey) node color-coding.]

[Figure 3: Recursion tree visualization for the Quick Sort algorithm on the array [25, 10, 40, 22, 35, 50], with tooltip showing pivot selection rationale.]

[Figure 4: Recursion tree visualization for the Binary Search algorithm looking for target value 18, with side panel description of the current interval halving step.]

[Figure 5: Recursion tree visualization for the Closest Pair of Points algorithm with points (2,3), (12,30), (40,50), (5,1), (12,10).]

[Figure 6: Recursion tree visualization for Strassen's Matrix Multiplication showing the 7-subproblem (M1–M7) decomposition at each recursive level.]

3.1 Explanation of the Design Figure

The architecture diagram organises the Recursion Tree Visualizer into three horizontal layers, each representing a distinct tier of the application's internal structure. Data flows vertically between layers via solid arrows, while internal module communication within the Application Logic Layer is shown using dashed arrows. A feedback loop from the Rendering Layer back to the browser client represents the final visual output the user observes.

Layer 1 — User Interface

The topmost layer contains all components through which the user directly interacts with the tool. It now comprises five modules:

- Algorithm Selector — a dropdown allowing the user to choose between Merge Sort, Quick Sort, Binary Search, Closest Pair of Points, and Strassen's Matrix Multiplication.
- Input Panel — where the user provides array inputs, search values, coordinate points, or matrix dimensions depending on the selected algorithm.
- Navigation Controls — Prev, Next, and Reset buttons that drive step-by-step traversal.

- Side Panel / Tooltip Display — a new component that displays a contextual description of the currently active recursive call, explaining its purpose, the subproblem it is solving, and how it relates to the parent call in the recursion tree.
- Display Canvas — the SVG viewport in which the recursion tree is rendered and animated with smooth transitions and color-coded node states.

Layer 2 — Application Logic

The middle layer is the computational core of the application. It contains five algorithm engines (Merge Sort, Quick Sort, Binary Search, Closest Pair, and Strassen), each of which generates tree node objects and passes them to the Tree State Manager. The Tree State Manager stores the ordered sequence of recursive calls, tracks the current step index, exposes navigation functions, and now additionally stores a descriptive annotation string for each node to be rendered in the side panel.

Layer 3 — Rendering and Output

The bottommost layer translates abstract tree data into a visual representation. The Tree Layout Engine calculates node coordinates. The Node Renderer draws nodes and edges onto the SVG canvas. The Step Animator highlights the currently active node using blue, marks visited nodes in green, and leaves pending nodes in grey, applying smooth CSS transitions between state changes. The annotation from the Tree State Manager is forwarded to the Side Panel component for display alongside the tree.

4. Module Description

The Recursion Tree Visualizer is composed of seven distinct software modules, each with a clearly defined purpose, set of responsibilities, and set of interactions with other modules.

4.1 Module 1: Algorithm Selector

Attribute	Details
Type	UI Control Component
Interface element	Dropdown menu (HTML <select>)
Current options	Merge Sort Quick Sort Binary Search Closest Pair of Points Strassen's Matrix Multiplication
Interacts with	Tree State Manager, Algorithm Engines, Side Panel

The Algorithm Selector module is the entry point of the application. When a selection is made, it fires a change event that clears all stored state, resets the step index to zero, and activates the corresponding algorithm engine. It is architecturally designed as a simple stateless UI control that delegates all state management to the Tree State Manager. This design keeps the codebase DRY and makes it straightforward to add future algorithms.

4.2 Module 2: Merge Sort Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Integer array provided via the Input Panel
Output	Ordered sequence of tree node objects (array segments, depth level, merge results, annotations)
Time complexity	$O(n \log n)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Merge Sort Engine implements the recursive divide-and-conquer logic of Merge Sort. At each recursive call, the engine generates a tree node object containing the array segment, recursion depth level, and an annotation string (e.g., "Splitting [38, 27, 43] into left half [38, 27] and right half [43]") passed to the Side Panel. The binary branching structure produces $\log n$ levels, clearly demonstrating $O(n \log n)$ complexity.

4.3 Module 3: Quick Sort Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Integer array provided via the Input Panel
Output	Ordered sequence of tree node objects (array partitions, pivot values, annotations)
Time complexity	Average $O(n \log n)$, worst case $O(n^2)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Quick Sort Engine implements pivot-based recursive partitioning. Each node annotation explains the pivot selection and partition rationale (e.g., "Pivot selected: 40. Partitioning array into elements less than 40 and greater than 40"). The resulting recursion tree illustrates how partition balance influences the algorithm's depth and complexity.

4.4 Module 4: Binary Search Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Sorted integer array and target value provided via the Input Panel
Output	Ordered sequence of tree node objects (search interval, midpoint, annotations)
Time complexity	$O(\log n)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Binary Search Engine implements the recursive divide-and-conquer strategy for searching within a sorted array. Each node annotation describes the current interval halving decision (e.g., "Midpoint: 16. Target 18 > 16, so searching right half [18, 21, 24]"). The recursion tree has logarithmic depth, clearly illustrating $O(\log n)$ complexity.

4.5 Module 5: Closest Pair Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	List of coordinate pairs (x, y) provided via the Input Panel
Output	Ordered sequence of tree node objects (point subsets, distance calculations, annotations)
Time complexity	$O(n \log n)$

Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)
----------------	---

The Closest Pair Engine implements the divide-and-conquer approach for the closest pair of points problem. Each node annotation explains the geometric step (e.g., "Dividing point set at $x = 12$. Left subset: 3 points. Right subset: 3 points. Computing closest pair in each half."). It recursively splits the point set, computes closest pairs in each half, and evaluates potential cross-border pairs within a bounded strip.

4.6 Module 6: Strassen's Matrix Multiplication Engine

Attribute	Details
Type	Algorithm Engine / Recursive Computation Module
Input	Two $n \times n$ matrices (n must be a power of 2) provided via the Input Panel
Output	Ordered sequence of tree node objects (sub-matrix blocks M1–M7, recursion depth, annotations)
Time complexity	$O(n^{2.807})$ — Strassen's complexity via Master Theorem
Recurrence	$T(n) = 7T(n/2) + O(n^2)$
Interacts with	Tree State Manager (emits nodes); Algorithm Selector (activated by)

The Strassen Engine implements Volker Strassen's 1969 algorithm for matrix multiplication. Instead of computing 8 recursive sub-matrix products (as in the naive divide-and-conquer approach), Strassen's algorithm computes only 7 products (M1 through M7) using cleverly constructed linear combinations of submatrices. This reduces the recurrence from $T(n) = 8T(n/2) + O(n^2)$ to $T(n) = 7T(n/2) + O(n^2)$, which by the Master Theorem resolves to $O(n^{\log_2 7}) \approx O(n^{2.807})$.

When invoked, the algorithm performs the following steps:

6. Divide each input matrix into four $n/2 \times n/2$ sub-matrices ($A_{11}, A_{12}, A_{21}, A_{22}$) and ($B_{11}, B_{12}, B_{21}, B_{22}$).
7. Compute 7 intermediate products $M1 = (A_{11} + A_{22})(B_{11} + B_{22})$, $M2 = (A_{21} + A_{22})B_{11}$, $M3 = A_{11}(B_{12} - B_{22})$, $M4 = A_{22}(B_{21} - B_{11})$, $M5 = (A_{11} + A_{12})B_{22}$, $M6 = (A_{21} - A_{11})(B_{11} + B_{12})$, $M7 = (A_{12} - A_{22})(B_{21} + B_{22})$.
8. Combine the 7 products to reconstruct the four quadrants of the result matrix C .
9. Recurse until base case (1×1 matrices) is reached.

Each recursive call generates a tree node object containing the sub-matrix dimensions, the current product label (M1–M7), recursion depth, and an annotation string explaining the semantic purpose of that product computation (e.g., "Computing $M1 = (A_{11} + A_{22})(B_{11} + B_{22})$. This product contributes to both diagonal quadrants C11 and C22 of the result."). The recursion tree branches 7-way at each level, producing a visually striking tree that makes the sub-cubic complexity immediately apparent. Students can directly compare this tree against the Merge Sort tree and observe why a branching factor of 7 versus 2 leads to dramatically different asymptotic behavior.

4.7 Module 7: Tree State Manager

Attribute	Details
Type	Central State Coordinator
Stores	Ordered array of all tree node objects (including annotations); current step index
Exposes	next(), prev(), reset(), getCurrentNode(), getFullTree(), getCurrentAnnotation()
Interacts with	Algorithm Engines, Navigation Controls, Rendering Module, Side Panel

The **Tree State Manager** is the central coordination module of the **Recursion Tree Visualizer**. When an algorithm engine executes, it passes each node object — now including an annotation string — into the **Tree State Manager**, which appends them to an ordered sequence following depth-first traversal. On every state change, it notifies both the **Rendering Module** (providing the full tree structure and currently active node) and the **Side Panel** component (providing the annotation for the current step). This event-driven design keeps the rendering layer and annotation layer independent from the algorithm logic.

4.8 Module 8: Rendering Module

Attribute	Details
Type	Visual Output Module
Subcomponents	Tree Layout Engine, Node Renderer, Step Animator
Input	Tree data, active node, and annotations from Tree State Manager
Output	Live SVG visualization with color-coded nodes and smooth transitions
Interacts with	Tree State Manager, Display Canvas, Side Panel

The **Rendering Module** produces the visual recursion tree displayed in the browser. The **Step Animator** applies a consistent three-state color-coding scheme: active nodes (currently executing call) are highlighted in blue, visited nodes (completed calls) are rendered in green, and pending nodes (calls not yet reached) are shown in grey. Smooth CSS transitions are applied between these state changes, providing visual continuity as the user steps through the algorithm. This visual differentiation allows learners to instantly understand the execution state of the entire recursion tree at any given step.

5. Dataset and Method Description

5.1 Dataset Description

The Recursion Tree Visualizer is an algorithm analysis and education tool. It does not operate on a fixed external dataset; instead, its input is user-defined at runtime and constitutes synthetic, user-generated data supplied interactively through the browser interface.

5.1.1 Input Specification: Merge Sort

Property	Description
Source	User-provided at runtime via the Input Panel
Format	A one-dimensional array of integers entered as a comma-separated list
Size / Scale	Array length n ; recursion tree depth grows approximately as $\log_2 n$
Preprocessing	Input validation to ensure all values are integers; recursive splitting into two equal halves
Example input	$A = [38, 27, 43, 3, 9, 82, 10]$
External dataset	None

5.1.2 Input Specification: Quick Sort

Property	Description
Source	User-provided at runtime via the Input Panel
Format	A one-dimensional integer array
Size / Scale	Array length n ; recursion depth varies depending on pivot selection
Preprocessing	Pivot selection; partition into elements smaller and larger than the pivot
Example input	$A = [25, 10, 40, 22, 35, 50]$
External dataset	None

5.1.3 Input Specification: Binary Search

Property	Description
Source	User-provided at runtime via the Input Panel
Format	Sorted integer array and target value
Size / Scale	Array length n ; recursion tree depth approximately $\log_2 n$

Preprocessing	Ensure the array is sorted; initialize search bounds low = 0, high = n - 1
Example input	A = [4, 8, 12, 16, 18, 21, 24], Target = 18
External dataset	None

5.1.4 Input Specification: Closest Pair of Points

Property	Description
Source	User-provided via the Input Panel
Format	A list of coordinate pairs (x, y)
Size / Scale	Number of points n; recursion tree depth approximately $\log_2 n$
Preprocessing	Points sorted by x-coordinate; recursively divided into left and right subsets
Example input	Points = {(2,3), (12,30), (40,50), (5,1), (12,10)}
External dataset	None

5.1.5 Input Specification: Strassen's Matrix Multiplication

Property	Description
Source	User-provided via the Input Panel
Format	Two $n \times n$ integer matrices; n must be a power of 2 (e.g., 2, 4, 8)
Size / Scale	Matrix dimension n; recursion tree depth $\log_2 n$; branching factor 7
Preprocessing	Validate that both matrices are square and of identical dimension; pad to next power of 2 if needed
Example input	A = [[1,2],[3,4]], B = [[5,6],[7,8]]
External dataset	None

5.2 Method Description

This section describes the algorithmic methods implemented in the Recursion Tree Visualizer, the rationale for their selection, the tree generation method used to produce the step sequence, and the rendering method used to lay out and display the recursion tree.

5.2.1 Method 1: Merge Sort

Merge Sort recursively divides the input array into two halves until each subarray contains a single element. These subarrays are then merged in sorted order during the combine phase. Recurrence: $T(n) = 2T(n/2) + O(n)$. Time complexity: $O(n \log n)$. The recursion tree forms a balanced binary tree with $\log_2 n$ levels.

5.2.2 Method 2: Quick Sort

Quick Sort selects a pivot element and partitions the array into two subsets — elements smaller and larger than the pivot — then sorts each partition recursively. Recurrence (average): $T(n) = 2T(n/2) + O(n)$. Time complexity: $O(n \log n)$ average, $O(n^2)$ worst case. The recursion tree illustrates how pivot balance influences depth and complexity.

5.2.3 Method 3: Binary Search

Binary Search repeatedly halves the search interval by comparing the target value with the midpoint element. Recurrence: $T(n) = T(n/2) + O(1)$. Time complexity: $O(\log n)$. The recursion tree has logarithmic depth and a single branch at each level, illustrating how repeated problem size reduction results in highly efficient algorithms.

5.2.4 Method 4: Closest Pair of Points

The divide-and-conquer algorithm sorts points by x-coordinate, divides the set into two halves, recursively computes the closest pair in each half, and examines candidate pairs across the dividing line. Recurrence: $T(n) = 2T(n/2) + O(n)$. Time complexity: $O(n \log n)$.

5.2.5 Method 5: Strassen's Matrix Multiplication

Strassen's algorithm divides each input matrix into 2×2 block form and computes 7 recursive products (M1–M7) instead of the naive 8, using carefully designed linear combinations of the input submatrices. The 7 products are then combined to reconstruct the output matrix. Recurrence: $T(n) = 7T(n/2) + O(n^2)$. Time complexity: $O(n^{\log_2 7}) \approx O(n^{2.807})$. The recursion tree has branching factor 7 and depth $\log_2 n$. This method is pedagogically significant because it demonstrates that the number of recursive subproblems directly determines the dominant complexity term — reducing subproblems from 8 to 7 reduces the exponent from 3 to approximately 2.807.

5.2.6 Tree Generation Method

All algorithm engines use a common tree generation method. Each recursive call produces a tree node object that records: unique node ID, recursion depth, parent node reference, input parameters, intermediate values, return value, and an annotation string describing the semantic purpose of this recursive call. Nodes are appended to the Tree State Manager in depth-first traversal order, which matches the exact execution order of recursive calls. This design allows the visualization to replicate the actual runtime behavior of the algorithm.

5.2.7 Rendering and Layout Method

The Tree Layout Engine computes node positions using a recursive layout algorithm inspired by the Reingold–Tilford tree layout method. Each node's vertical position is determined by its recursion depth, while horizontal positions are calculated to maintain balanced subtree spacing. The Node Renderer draws nodes and edges using SVG

graphics. The Step Animator applies a three-state color-coding scheme (active: blue; visited: green; pending: grey) and smooth CSS transitions between recursion steps. The annotation for the current node is forwarded to the Side Panel component, which renders it alongside the tree to explain the purpose of each recursive call.

6. Conclusion

The Recursion Tree Visualizer was developed in response to a clearly identified gap in algorithm education: the absence of a purpose-built, web-accessible, step-by-step animated recursion tree tool for canonical divide-and-conquer algorithms taught in undergraduate DAA curricula. This report has documented the project across several dimensions: its introduction and objectives, its literature foundations, its system architecture, its internal module structure, and its algorithmic and methodological design.

6.1 Summary of Achievements

The project successfully delivers on its primary objectives by implementing a web-based visualization tool capable of rendering the step-by-step execution of five classical algorithms as animated recursion trees: Merge Sort, Quick Sort, Binary Search, the Closest Pair of Points algorithm, and Strassen's Matrix Multiplication.

Three significant enhancements distinguish this version from the original abstract:

- Contextual annotation system: Each recursive step is now accompanied by a tooltip and side panel description explaining the purpose of that specific call — why the subproblem exists, what it computes, and how it contributes to the final solution. This moves the tool beyond value display toward genuine conceptual scaffolding.
- Color-coded node states with smooth transitions: Active nodes are highlighted in blue, visited nodes in green, and pending nodes in grey. Smooth CSS transitions between state changes provide visual continuity and allow learners to track execution state across the entire tree simultaneously.
- Strassen's Matrix Multiplication: The addition of this algorithm introduces a branching factor of 7 and sub-cubic complexity $O(n^{2.807})$, offering a fundamentally different recursive structure that contrasts sharply with the binary and logarithmic recursion trees of the other four algorithms.

The modular three-layer architecture (User Interface, Application Logic, and Rendering) ensures a clean separation of concerns and makes the system highly extensible for future algorithm additions.

6.2 Limitations

Despite its contributions, several limitations of the current implementation should be acknowledged. First, although the system now includes five algorithms with annotations, the overall range remains limited compared to the full breadth of divide-and-conquer algorithms taught in DAA courses. Second, the tool has not yet undergone formal empirical evaluation in a classroom environment. Third, the system lacks persistent session state — refreshing the browser resets the current recursion tree position. Fourth, for very large inputs, the recursion tree can become visually crowded; adaptive zoom and pan controls are not yet fully implemented.

6.3 Future Work

Several promising directions exist for extending the capabilities of the Recursion Tree Visualizer:

10. Integration of additional algorithms such as Tower of Hanoi, Fibonacci recursion, Karatsuba Multiplication, and additional computational geometry algorithms.
11. An interactive complexity analysis panel that dynamically displays the recurrence relation and its Master Theorem resolution alongside the visual tree.
12. A formal classroom evaluation study comparing student performance on recursion problems before and after exposure to the visualization tool.
13. Persistent session state using browser storage so users can resume a walkthrough after a page refresh.
14. Adaptive zoom and pan controls for large recursion trees, particularly relevant for Strassen's algorithm on larger matrices.

6.4 Closing Remarks

The Recursion Tree Visualizer demonstrates that a focused, purpose-built educational tool — one designed specifically to visualize recursive algorithms with contextual annotations, smooth transitions, and clear color-coded node states — can effectively address a meaningful gap in algorithm education. By making the execution of key divide-and-conquer algorithms visible, interactive, and analytically transparent, the tool provides a learning experience that complements traditional instructional methods such as lectures, textbooks, and static diagrams.

The addition of Strassen's Matrix Multiplication, contextual step descriptions, and enhanced visual differentiation in this revised version demonstrates the tool's capacity to grow in both breadth and pedagogical depth. As the platform continues to evolve, it has the potential to become a valuable reference tool for students studying recursion and divide-and-conquer algorithms within undergraduate computer science curricula.